Colégio Técnico – UFMG
Algoritmos e Estrutura de Dados

Relatório As-Built

Sistema de Controle de Tráfego Aéreo e Pistas de Pouso

Este documento descreve, de forma transparente, a arquitetura de dados efetivamente construída e testada para o protótipo do Controlador Aéreo Nacional, contemplando os três componentes exigidos: (1) Aproximação e Autorização de Pouso, (2) Circulação e Gerenciamento de Solo (Taxiways) e (3) Despacho de Porão, Cadastro Prévio e Inspeção.

A. Descrição da Arquitetura

Para cada um dos três componentes do sistema, apresenta-se a estrutura de dados escolhida, a justificativa técnica da decisão, os dados básicos armazenados, os benefícios práticos obtidos e o tratamento das peculiaridades operacionais do domínio.

A.1. Componente 1 — Aproximação e Autorização de Pouso

Mapeamento do problema

A regra operacional exige que uma aeronave de nível de prioridade numericamente inferior (Emergência = 1, Comercial = 2, Carga = 3) seja sempre autorizada a pousar antes de uma de nível superior, independentemente da ordem de chegada. Isso caracteriza uma **fila de prioridade**. Optou-se por uma **lista encadeada simples mantida sempre ordenada por prioridade no momento da inserção** (inserção ordenada), em vez de um vetor ou de uma fila circular. Como o número de aeronaves aguardando pouso é imprevisível, uma lista encadeada evita redimensionamento de vetor e desperdício de memória; e como a política de atendimento é fixa (3 níveis) e a operação mais frequente é remover a maior prioridade, manter a ordenação na inserção transfere o custo de ordenação (O(n) no pior caso) para o momento de chegada da aeronave — que é menos frequente e menos crítico em tempo — e deixa a autorização de pouso, a operação mais sensível ao tempo, em O(1).

Dados básicos da estrutura

- Nó `Aviao`: nome (string), prioridade (enum: Emergência/Comercial/Carga), id_aviao (inteiro) e ponteiro `prox` para o próximo nó.
- Controle `Avioes`: ponteiros `inicio` e `fim` da lista e contador `tam`.

Benefícios e vantagens práticas

- Autorização de pouso (operação mais frequente e mais crítica) é O(1): basta remover o nó `inicio`.
- Sem limite fixo de aeronaves em espera e sem realocação de memória, ao contrário de um vetor.
- Ponteiro `fim` permite inserção rápida quando a nova aeronave é a de menor prioridade (caso comum em picos de tráfego comercial).

Tratamento específico do domínio

Dentro de um mesmo nível de prioridade, a inserção ordenada preserva a ordem relativa de chegada (estabilidade), pois um novo item só é inserido depois do último elemento de prioridade igual ou menor — ou seja, entre duas aeronaves comerciais, pousa primeiro quem chegou primeiro. O caso de fila vazia e o caso de remoção que esvazia a fila (atualização de `fim` para NULL) são tratados explicitamente.

A.2. Componente 2 — Circulação e Gerenciamento de Solo (Taxiways)

Mapeamento do problema

O enunciado descreve as taxiways como uma **rotatória**: não há início nem fim físico, e a aeronave circula continuamente até liberar o portão. Isso mapeia diretamente para uma **lista circular**. Como o sistema precisa navegar em sentido horário (padrão) e, em certas condições, em sentido anti-horário (quando o portão está logo atrás e o caminho está livre), a lista precisa ser **duplamente encadeada** — sem o ponteiro reverso (`ant`), percorrer o circuito no sentido anti-horário exigiria varrer toda a lista a partir da referência, o que é ineficiente.

Dados básicos da estrutura

- Nó `NoTaxiway`: id da posição física, ponteiro `ocupante` para o avião presente (ou NULL), e ponteiros `prox`/`ant`.

- Cada avião (`Aviao`) guarda apenas nome, id, o id do portão de destino e um ponteiro para o nó onde está posicionado — não guarda o caminho percorrido.
- `Circuito`: um ponteiro de referência qualquer (o círculo não tem início) e a lista dos aviões atualmente em solo.

Benefícios e vantagens práticas

- Entrada e saída de uma aeronave do circuito (pouso/atracagem) é O(1) e não requer inserir ou remover nós: apenas o campo `ocupante` do nó muda, então a circulação das demais aeronaves nunca é interrompida — exatamente a exigência do enunciado.
- A lista circular elimina a necessidade de tratar caso especial de 'fim de lista': o próximo nó sempre existe.
- O ponteiro duplo permite decidir o sentido de deslocamento em O(1) por passo, sem re-percorrer a estrutura.

Tratamento específico do domínio

A escolha de sentido é resolvida a cada passo de simulação: calcula-se a distância até o portão de destino andando por `ant`; se essa distância for pequena (constante `LOGO_ATRAS`) e o caminho estiver livre de outras aeronaves, o avião usa o sentido anti-horário — caso contrário, usa o sentido horário padrão. Colisões são evitadas verificando se o próximo nó já está ocupado antes de mover; quando duas aeronaves se aproximam em sentidos opostos no mesmo trecho, cada uma simplesmente aguarda quando o próximo nó está ocupado, o que produz alternância natural de avanço a cada rodada de simulação sem necessidade de uma estrutura de bloqueio adicional.

A.3. Componente 3 — Despacho de Porão, Cadastro Prévio e Inspeção

Mapeamento do problema

Este componente reúne dois sub-problemas com naturezas diferentes, tratados com duas estruturas distintas. O **cadastro prévio (manifesto)** precisa de consulta arbitrária por id e remoção administrativa antes do embarque — operações de uma **lista encadeada simples**. Já o **porão físico** é explicitamente restrito pelo enunciado: 'o operador só tem acesso imediato ao último contêiner ou bagagem que foi embarcado' — essa é, por definição, uma disciplina **LIFO (Last In, First Out)**, ou seja, uma **pilha**.

Dados básicos da estrutura

- Nó `ItemManifesto` (lista): id, dono, peso, status (Cadastrado/Embarcado/Descarregado/Isolado/Removido) e `prox`.
- Nó `NoPorao` (pilha): id, dono, peso e `prox` apontando para o item imediatamente abaixo.
- `SistemaBagagem`: ponteiros para o início/fim do manifesto e para o `topo` da pilha do porão.

Benefícios e vantagens práticas

- Empilhar (embarque) e desempilhar (descarregamento com double-check) são O(1), refletindo fielmente a restrição mecânica real do porão.
- O manifesto, por ser uma lista independente da pilha, permite consultar ou remover administrativamente qualquer item por id sem tocar na pilha física — as duas operações não competem pela mesma estrutura.
- Separar as duas estruturas evita duplicar regras: o status de cada item vive só no manifesto; o porão guarda apenas o necessário para conferência física.

Tratamento específico do domínio — double-check e interceptação

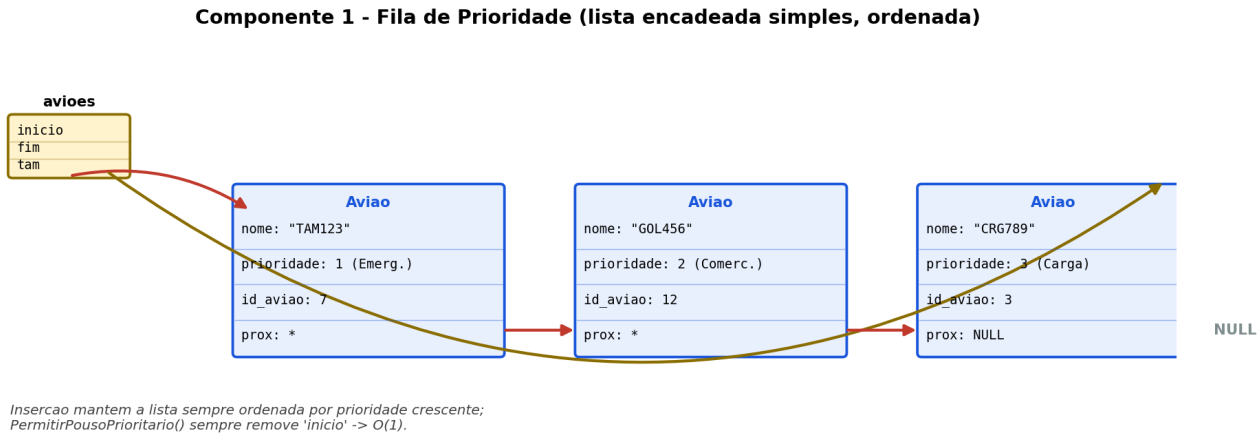
A cada `DescarregarProximo()`, o item retirado do topo da pilha é conferido contra o registro de mesmo id no manifesto (dono e peso); ausência do id ou divergência de dados gera um alerta imediato, sem interromper o descarregamento dos demais itens.

Para o procedimento de interceptação de emergência (remover uma bagagem específica que está no meio da pilha, sem alterar a ordem das demais), utiliza-se uma **pilha auxiliar**: desempilha-se o porão item a item, empilhando cada um na auxiliar, até localizar o alvo (que é isolado e não retorna); em seguida, a pilha auxiliar é devolvida ao porão. Como cada transferência entre pilhas inverte a ordem uma vez, duas transferências consecutivas restauram a ordem original das bagagens remanescentes, satisfazendo exatamente os passos A, B e C exigidos no enunciado sem precisar de nenhuma estrutura auxiliar além de uma segunda pilha.

B. Diagramas de Memória e Representação dos Nós

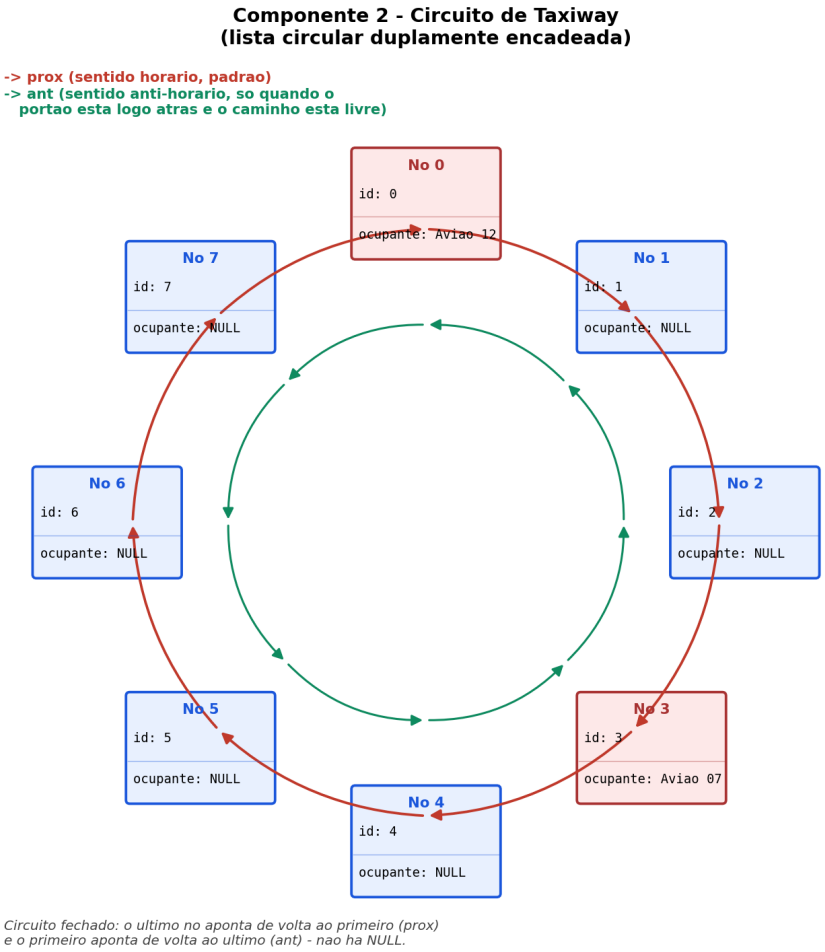
Os diagramas a seguir ilustram o estado da memória durante a execução do programa, destacando os nós, seus campos de dados e os ponteiros de controle, para um cenário de exemplo de cada componente.

B.1. Componente 1 — fila de prioridade (lista simples ordenada)



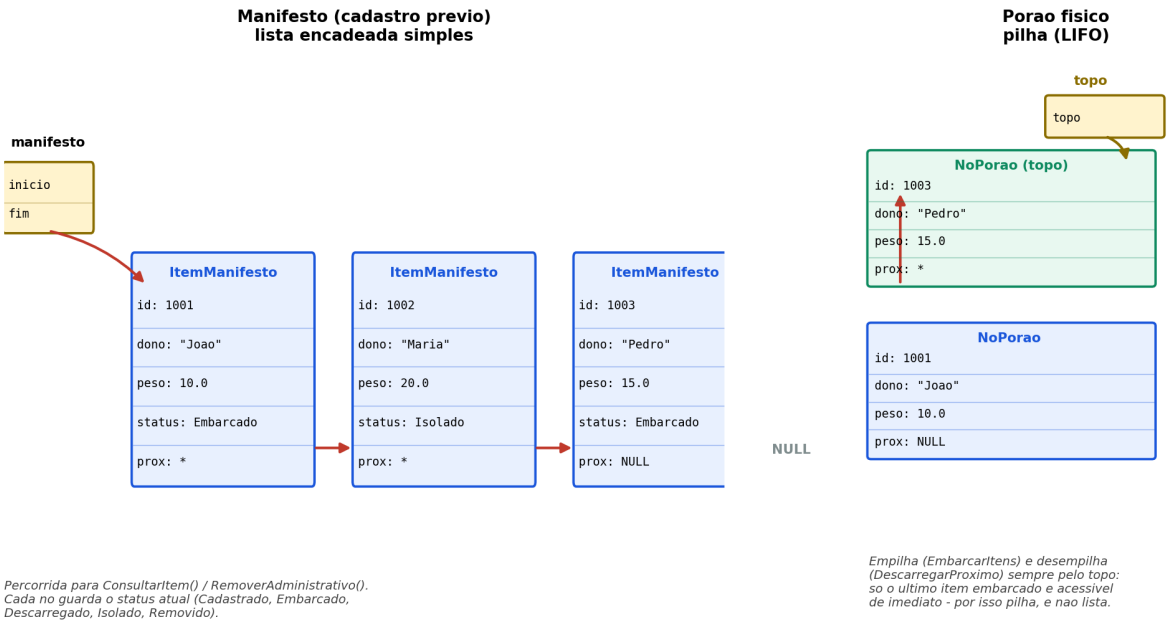
Estado de memória com três aeronaves aguardando pouso, já ordenadas por prioridade crescente. As setas vermelhas curvas partem da estrutura de controle (`inicio`/`fim`) para o primeiro e o último nó; as setas retas representam o campo `prox` de cada nó.

B.2. Componente 2 — circuito de taxiway (lista circular dupla)



Circuito com 8 nós. Nós em vermelho estão ocupados por uma aeronave (campo `ocupante` *preenchido). As setas externas (vermelhas) representam o encadeamento* `prox` *em sentido horário; as internas (verdes) representam* `ant`*, em sentido anti-horário. Por ser circular, não existe ponteiro NULL.*

B.3. Componente 3 — manifesto (lista) e porão (pilha)



Percorrida para ConsultarItem() / RemoverAdministrativo(). Cada no guarda o status atual (Cadastrado, Embarcado, Descarregado, Isolado, Removido).

Empilha (EmbarcarItens) e desempilha (DescarregarProximo) sempre pelo topo: so o ultimo item embarcado e acessivel de imediato - por isso pilha, e nao lista.

À esquerda, o manifesto como lista encadeada simples, com o item 1002 já marcado como `Isolado` *pela inspeção de emergência. À direita, o estado do porão físico como pilha após a mesma inspeção: o item 1002 foi removido do meio da estrutura e a ordem relativa dos itens 1003 (topo) e 1001 foi preservada, exatamente como restaurado pelo algoritmo de pilha auxiliar descrito na Seção A.3.*